

React Fiber Overview

A **Fiber** is a **JavaScript object** that represents a **React element's work unit** — a component, DOM node, or text node.

Each Fiber node is part of a **linked tree structure** (like a linked list + tree hybrid).

Think of each component (function, class, or DOM tag) as a **task**.

React creates a **Fiber** for that task, which stores:

- What this component is
- Its props & state
- Its children
- What changed since last render
- What to do next (update, delete, or add)

So, each `<div>`, `<App>`, `<Button>` etc. in your app has a **Fiber Node** behind it.

Two Key Phases in the Fiber Reconciliation Process:

React Fiber introduces a **two-phase architecture** for rendering:

1. Render Phase

- React **builds the Fiber tree**
- Calculates changes (diffing old vs new)
- This phase **can be paused, aborted, or resumed**
- It's **pure and side-effect-free**

Think of it as *“planning what to update”*.

2. Commit Phase

- React applies changes to the **real DOM**
- Runs lifecycle methods (`componentDidMount`, `useEffect`)
- This phase **is synchronous** — once it starts, it can't be stopped

Think of it as “*performing the updates*”.

When you write React code like this:

```
function App() {  
  return <h1>Hello React</h1>;  
}
```

React doesn't just render it to the screen.

Behind the scenes, it builds a *special data structure* called the **Fiber Tree** — this is how React keeps track of *what to render*, *what changed*, and *what needs updating*.

Inside a Fiber Node:

```
const fiberNode = {  
  
  type: App,           // The React component (function or class)  
  
  key: null,           // For lists/reconciliation  
  
  stateNode: null,     // Reference to DOM node or class instance  
  
  return: parentFiber, // Parent Fiber node  
  
  child: childFiber,   // First child  
  
  sibling: siblingFiber, // Next sibling  
  
  pendingProps: props, // Props for the next render  
  
  memoizedProps: prevProps, // Props from previous render  
  
  memoizedState: state,  // State after last render
```

```
flags: Update,      // Type of work to do (Placement, Update, Deletion)
alternate: oldFiber, // Used for diffing between renders
};
```

Hooks like `useState` or `useEffect` are stored inside each Fiber Node in a *linked list*.

`Fiber.memoizedState` → Hook1 → Hook2 → Hook3 → null

Each Hook object stores:

- **memoizedState** → the current value
- **queue** → list of pending updates
- **next** → next hook in the chain